SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

COMPUTER SCIENCE AND ENGINEERING PROGRAMME

CSA3717- SOFTWARE TESTING

EXPERIMENT 9

Case Study: Banking Application – Negative Test Scenarios and Test Cases

Aim:

To design and execute negative test scenarios and test cases for a Banking Application to verify that invalid transactions, incorrect inputs, and unauthorized operations are properly rejected.

Problem Description

A banking application allows customers to securely log in, view account balances, transfer funds, make permitted transactions, and view transaction history. The application must validate credentials, transaction limits, account balance, beneficiary details, and transaction status. The testing team is responsible for verifying whether the application works correctly under valid and invalid conditions. They must prepare positive and negative test cases and document any defects identified during testing.

Functional Requirements

1. Customers must log in with valid credentials.

2. Account balance must be displayed correctly.

3. Valid fund transfers must be processed.

4. Transfers must not exceed available balance or configured limits.

5. Valid beneficiary and transaction details must be accepted.

6. Transaction confirmation requires successful authorization.

7. Invalid inputs must display appropriate error messages.

3. Negative Test Cases

Negative testing verifies how the application behaves when invalid, incomplete, or unexpected inputs are provided.

Test Case ID	Test Scenario	Test Input / Action	Expected Result	Status
NT01	Wrong login password	Correct ID and wrong password	Login denied	Pass
NT02	Blank login	Leave mandatory fields blank	Validation messages displayed	Pass
NT03	Transfer amount zero	Enter 0 as amount	Transaction rejected	Pass
NT04	Transfer more than balance	Enter amount greater than available balance	Transfer rejected	Pass
NT05	Invalid beneficiary	Enter invalid beneficiary details	Beneficiary rejected	Pass
NT06	Duplicate transaction	Submit the same transfer	Only one valid	Fail

	submission	twice rapidly	transaction should be processed	
NT07	Invalid transaction amount format	Enter letters/special invalid value	Input rejected	Pass
NT08	Access account without login	Open protected account page while logged out	Redirected to login	Pass

4. Detailed Execution of a Failed Test Case

Test Case NT06
Failed

Test scenario: Duplicate transaction submission

Precondition

A valid account and beneficiary exist and the account has sufficient balance.

Test steps

1. Login.
2. Enter a valid transfer.
3. Click Submit.
4. Immediately submit the same transfer again.
5. Check transaction history.

Expected result

The application should prevent duplicate processing and create only one transaction.

Actual result

The same transfer is processed twice.

Conclusion

The test fails because duplicate transaction protection is not enforced.

5. Sample Bug Report

A bug report is a document that records a defect discovered during software testing. It provides developers with sufficient information to reproduce, investigate, and fix the issue.

Field	Details
Bug ID	BR-001
Linked Test Case	NT06
Severity	High
Priority	High
Bug Title	Duplicate transaction submission
Environment	Application: Banking Application; Browser: Google Chrome; Operating System: Windows 10; Test Module: Relevant application module
Preconditions	A valid account and beneficiary exist and the account has sufficient balance.
Steps to Reproduce	1. Login. 2. Enter a valid transfer. 3. Click Submit. 4. Immediately submit the same transfer again.

	5. Check transaction history.
Expected Result	The application should prevent duplicate processing and create only one transaction.
Actual Result	The same transfer is processed twice.
Impact	The defect may cause incorrect system behavior, data inconsistency, transaction errors, or poor user experience.
Suggested Fix	Apply validation on the client and server side and repeat the validation at the final submission/transaction stage.
Status	Open — awaiting developer investigation and correction.

6. Bug Life Cycle

New — Tester reports the defect
Assigned — Developer receives the defect
In Progress — Developer investigates and fixes it
Fixed — Developer submits the correction
Retesting — Tester verifies the correction
Closed — Tester confirms the defect is resolved
If the defect still exists after the fix, the tester reopens the bug and sends it back for further correction.

7. Difference Between Positive and Negative Testing

Positive Testing	**Negative Testing**
Uses valid inputs.	Uses invalid or unexpected inputs.
Checks whether expected operations succeed.	Checks whether errors are handled correctly.
Example: valid login or valid transaction.	Example: wrong password or invalid transaction.
Expected result is successful execution.	Expected result is safe rejection or appropriate error handling.